

# UNIVERSIDAD TECNOLÓGICA LA SALLE (ULSA)

León, Nicaragua

## Sistemas de Identificación Automática

### Actividad 2 – Node-RED

#### Presentado por:

- Alfredo Benito Berríos Contreras.
- David Israel Salinas Fonseca.
- Axel Xavier Mercado Carmona.
- Osmar Alexander Salgado Osejo.
- Yerling Alexander Sandino Penado.

#### Revisado por:

- Ing. Herson Eduardo Guzmán Castillo.

León, domingo 12 de octubre de 2025

## Índice

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 1. Introducción .....                                                | 3  |
| 2. Objetivos .....                                                   | 4  |
| 1. General:.....                                                     | 4  |
| 2. Específicos:.....                                                 | 4  |
| 3. Desarrollo.....                                                   | 5  |
| 1. Configuración del Bróker MQTT: .....                              | 5  |
| 2. Montaje Físico y Programación del ESP32:.....                     | 6  |
| 3. Configuración del MQTT Explorer y comunicación con la ESP32 ..... | 10 |
| 4. Funcionamiento del proceso .....                                  | 12 |
| 5. Configuración del Node-RED .....                                  | 14 |
| 4. Resultados y Análisis .....                                       | 18 |
| 6. Análisis .....                                                    | 20 |
| 5. Conclusiones .....                                                | 21 |
| 6. Anexos.....                                                       | 22 |
| 7. Referencias bibliográficas .....                                  | 31 |

## 1. Introducción

El Internet de las Cosas (IoT) ha trascendido la conceptualización teórica para convertirse en una fuerza transformadora en múltiples industrias y en la vida cotidiana. La capacidad de interconectar dispositivos físicos a través de internet, permitiéndoles recopilar datos y actuar sobre ellos, ha abierto un abanico de posibilidades para la automatización, la eficiencia y la toma de decisiones informadas. En el corazón de esta revolución se encuentran tecnologías que facilitan una comunicación fluida y eficiente, como el protocolo MQTT, diseñado específicamente para entornos con recursos limitados y redes de baja fiabilidad, características comunes en proyectos de IoT.

En este contexto, herramientas como Node-RED emergen como catalizadores clave para el desarrollo y la creación de prototipos. Node-RED, con su enfoque de programación visual basada en flujos, democratiza el acceso a la creación de aplicaciones IoT complejas. Permite a los desarrolladores, ingenieros y entusiastas conectar hardware, APIs y servicios en línea de manera intuitiva, reduciendo drásticamente las barreras de entrada y acelerando el ciclo de desarrollo desde la idea hasta el prototipo funcional.

El sector agrícola, particularmente la gestión del riego es un campo fértil para la aplicación de estas tecnologías. El riego tradicional a menudo se basa en temporizadores fijos o intervención manual, lo que puede conducir a un uso ineficiente del agua, ya sea por exceso o por defecto, afectando la salud de los cultivos y desperdiciando un recurso vital. La implementación de un sistema de riego "inteligente" permite una gestión precisa y basada en datos, asegurando que el agua se aplique solo cuando y donde se necesita.

Este proyecto se sitúa en la intersección de estas tecnologías para abordar precisamente este desafío. A través de la integración de un microcontrolador ESP32, sensores de humedad, actuadores y la plataforma Node-RED, se ha desarrollado un prototipo de sistema de riego inteligente. Este sistema no solo automatiza el monitoreo de las condiciones del suelo, sino que

también proporciona una interfaz de control y visualización remota, demostrando una solución completa, de bajo costo y altamente efectiva.

El presente informe detalla el proceso completo de diseño, montaje y configuración de este sistema. Se abordan desde la configuración del broker MQTT local, que actúa como el sistema nervioso central de la comunicación, hasta el ensamblaje del circuito físico y la programación del ESP32. Finalmente, se culmina con la creación de un dashboard interactivo en Node-RED, validando la viabilidad de esta arquitectura para aplicaciones prácticas de IoT y sentando las bases para futuras mejoras y escalabilidad.

## **2. Objetivos**

### **1. General:**

Desarrollar un prototipo funcional de un sistema de riego inteligente utilizando un ESP32, que monitoree la humedad del suelo en tiempo real y permita la visualización y control de sus estados a través de un dashboard en Node-RED, utilizando el protocolo MQTT para la comunicación de datos.

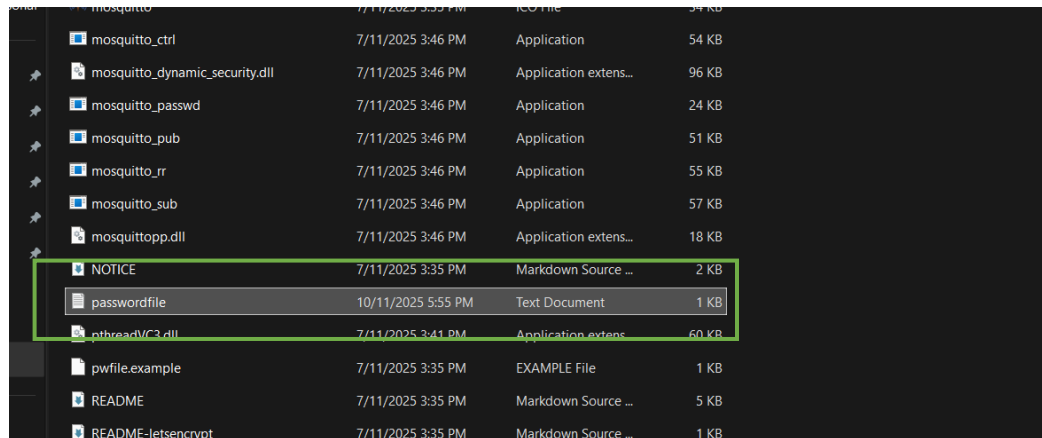
### **2. Específicos:**

1. Configurar un broker MQTT local (Mosquitto) para gestionar de forma segura y eficiente la comunicación entre el dispositivo ESP32 y la plataforma de visualización Node-RED.
2. Ensamblar y programar un circuito electrónico basado en un ESP32 que integre un sensor de humedad, LEDs, un pulsador y un buzzer para el control y la señalización local del estado del sistema.
3. Diseñar e implementar un flujo en Node-RED que se suscriba a los topics MQTT, procese los datos recibidos del ESP32 y los represente visualmente en un dashboard interactivo para el monitoreo y la comprensión del estado del sistema de riego.

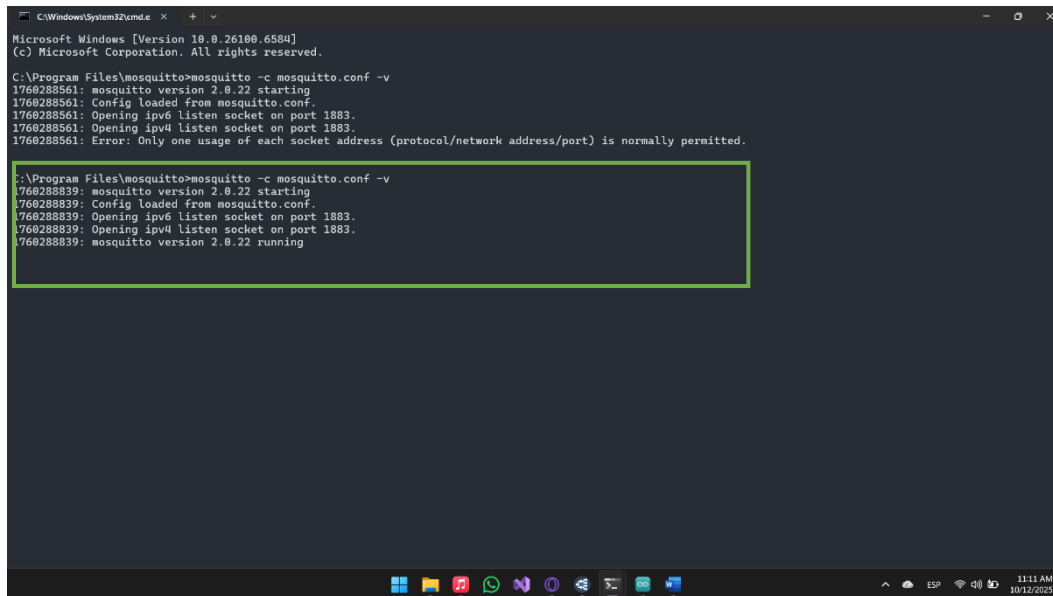
### 3. Desarrollo

#### 1. Configuración del Bróker MQTT:

Primero, se realizó la creación de un archivo de texto que almacena las credenciales con el siguiente comando ***mosquitto\_passwd -c passwordfile.txt Alfredo*** y como password **220705**.



Se inicia el bróker con nuestra configuración del **mosquitto.conf** desde la cmd.

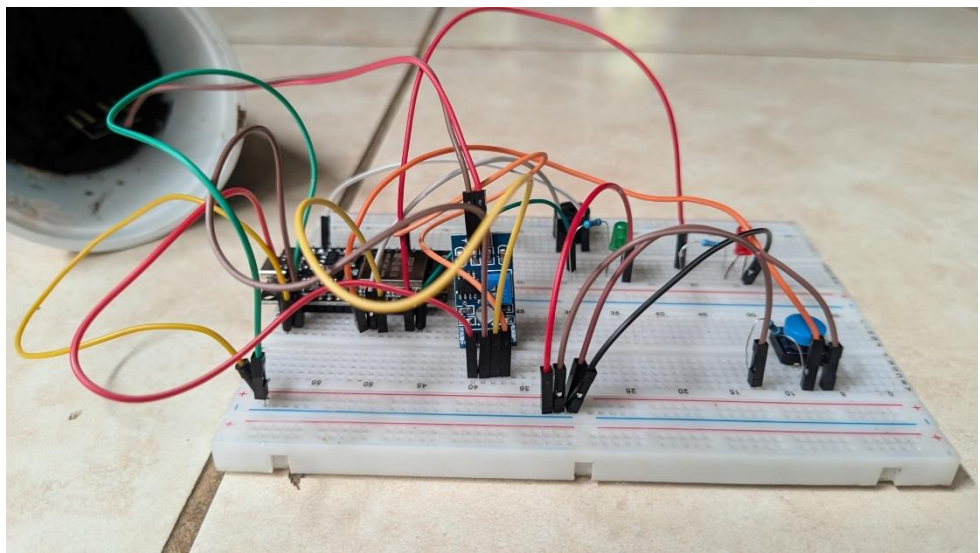
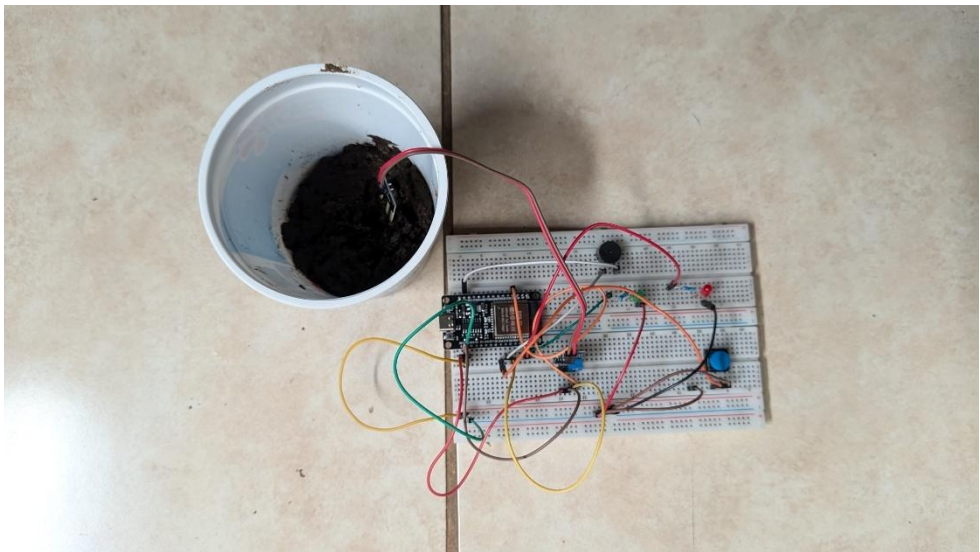


2. Montaje Físico y Programación del ESP32:

Componentes:

|    | A        | B                          | C |
|----|----------|----------------------------|---|
| 1  | Cantidad | Componentes                |   |
| 2  | 1        | Placa - ESP32              |   |
| 3  | 1        | Sensor de humedad de suelo |   |
| 4  | 1        | LED rojo                   |   |
| 5  | 1        | LED verde                  |   |
| 6  | 1        | Buzzer                     |   |
| 7  | 1        | Pulsador 4 pines           |   |
| 8  | 2        | Resistencia 110 ohms       |   |
| 9  | 1        | Resistencia 10K ohms       |   |
| 10 | 1        | Protoboard                 |   |
| 11 | 20       | Jumpers                    |   |
| 12 |          |                            |   |

| Esquema de conexión |                                                           |               |
|---------------------|-----------------------------------------------------------|---------------|
| Componente          | Conexión                                                  | Pin en ESP32  |
| Sensor de humedad   | Vcc, Gnd, Aout                                            | 3V3, Gnd, D34 |
| LED rojo            | Anodo a 110 ohms                                          | D22           |
| LED verde           | Anodo a 110 ohms                                          | D23           |
| Pulsador            | Pin a 3V3, pin opuesto a D18 y resistencia 10K ohms a Gnd | D18           |
| Buzzer              | Pin positivo                                              | D19           |
| Componentes a GND   | Catodos de LEDs y pin negativo del Buzzer                 | Gnd           |



## Código - ESP32

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

//DATOS DE CONEXIÓN
const char* ssid = "Fam_HERNANDEZ CONTRERAS";
const char* password = "NORTHLANE_2025";
const char* mqtt_server = "192.168.1.26";
const int mqtt_port = 1883;
const char* mqtt_user = "Alfredo";
const char* mqtt_pass = "220705";

//TOPICS MQTT
const char* topic_sensor = "zonal/riego/sensor";
const char* topic_led_rojo = "zonal/riego/estado/ledRojo";
const char* topic_led_verde = "zonal/riego/estado/ledVerde";
const char* topic_boton = "zonal/riego/estado/boton";
const char* topic_buzzer = "zonal/riego/estado/buzzer";

//DEFINICIÓN DE PINES
const int SENSOR_AOUT_PIN = 34;
const int LED_REGAR_PIN = 22;
const int LED_BOMBA_PIN = 23;
const int BOTON_BOMBA_PIN = 18;
const int BUZZER_PIN = 19;

//VARIABLES GLOBALES Y DE CALIBRACIÓN
const int VALOR_SECO = 4095;
const int VALOR_HUMEDO = 1700;
int valorHumedad = 0;
int porcentajeHumedad = 0;

//VARIABLES PARA DETECTAR CAMBIOS DE ESTADO
int last_led_rojo_state = -1;
int last_led_verde_state = -1;
int last_boton_state = -1;
int last_buzzer_state = -1;

// Intervalo de publicación del sensor
const long publishInterval = 2000;
unsigned long lastPublishTime = 0;

WiFiClient espClient;
PubSubClient client(espClient);

//FUNCIÓN DE RECONEXIÓN
void reconnect() {
    while (!client.connected()) {
```

```

Serial.print("Intentando conectar al broker MQTT...");
String clientId = "ESP32_Sensor_Riego-" + WiFi.macAddress();
clientId.replace(":", "");
if (client.connect(clientId.c_str(), mqtt_user, mqtt_pass)) {
    Serial.println("Conectado al broker MQTT");
} else {
    Serial.print("Fallo, rc=");
    Serial.print(client.state());
    Serial.println(" -> Reintentando en 5 segundos");
    delay(5000);
}
}
}

//FUNCIÓN PARA PUBLICAR CAMBIOS DE ESTADO
void publishState(const char* topic, int& lastState, int currentState) {
    if (currentState != lastState) {
        client.publish(topic, String(currentState).c_str(), true); // `true`
para mensaje retenido
        lastState = currentState;
    }
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_REGAR_PIN, OUTPUT);
    pinMode(LED_BOMBA_PIN, OUTPUT);
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(BOTON_BOMBA_PIN, INPUT);
    pinMode(SENSOR_AOUT_PIN, INPUT);
    digitalWrite(LED_REGAR_PIN, LOW);
    digitalWrite(LED_BOMBA_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi conectado");
    Serial.print("IP local: ");
    Serial.println(WiFi.localIP());
    client.setServer(mqtt_server, mqtt_port);
}

void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    //LECTURA Y LÓGICA DE CONTROL

```



```
valorHumedad = analogRead(SENSOR_AOUT_PIN);
porcentajeHumedad = map(valorHumedad, VALOR_SECO, VALOR_HUMEDO, 0, 100);
if (porcentajeHumedad > 100) porcentajeHumedad = 100;
if (porcentajeHumedad < 0) porcentajeHumedad = 0;

//Lógica de control local
int led_rojo_state = (porcentajeHumedad < 50) ? HIGH : LOW;
digitalWrite(LED_REGAR_PIN, led_rojo_state);

int boton_state = digitalRead(BOTON_BOMBA_PIN);
digitalWrite(LED_BOMBA_PIN, boton_state);

int buzzer_state = (porcentajeHumedad >= 80) ? HIGH : LOW;
digitalWrite(BUZZER_PIN, buzzer_state);

//PUBLICACIÓN DE ESTADOS
publishState(topic_led_rojo, last_led_rojo_state, led_rojo_state);
publishState(topic_boton, last_boton_state, boton_state);
publishState(topic_led_verde, last_led_verde_state, boton_state); // El
LED verde refleja el estado del botón
publishState(topic_buzzer, last_buzzer_state, buzzer_state);

//PUBLICACIÓN PERIÓDICA DEL SENSOR
if (millis() - lastPublishTime > publishInterval) {
  lastPublishTime = millis();
  StaticJsonDocument<128> doc;
  doc["humedad"] = porcentajeHumedad;
  doc["raw"] = valorHumedad;

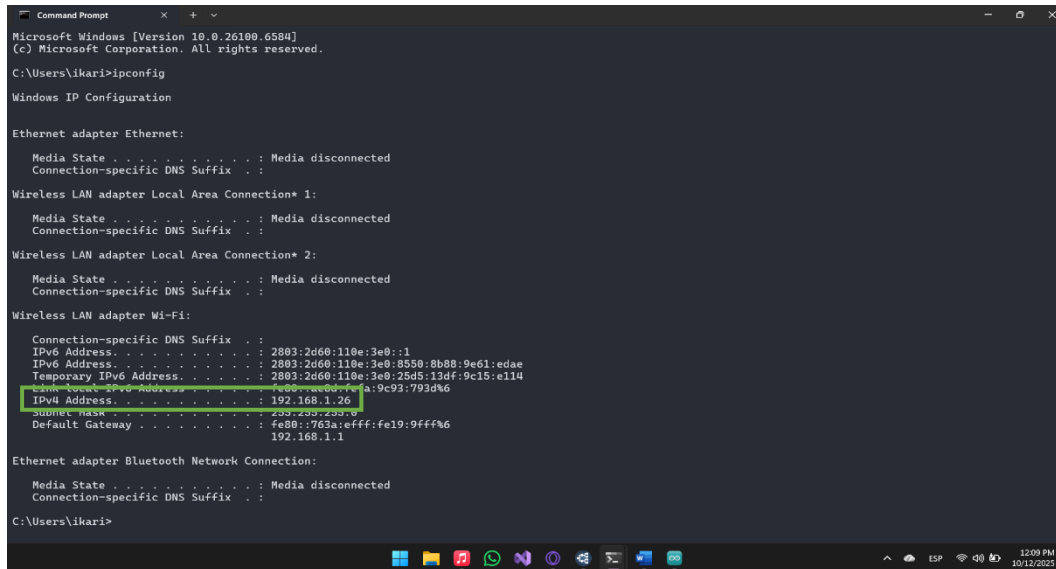
  char buffer[128];
  serializeJson(doc, buffer);
  client.publish(topic_sensor, buffer);

  Serial.print("Publicado en MQTT: ");
  Serial.println(buffer);
}
}
```

### 3. Configuración del MQTT Explorer y comunicación con la ESP32

Antes de construir el dashboard, es crucial verificar que el ESP32 se esté comunicando correctamente con el broker. MQTT Explorer es la herramienta perfecta para esta tarea, ya que nos permite "espiar" todos los mensajes que viajan por el bróker en tiempo real.

El ESP32 y MQTT Explorer necesitan la dirección IP de la computadora donde se está ejecutando el broker Mosquitto.



```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ikari>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Wireless LAN adapter Local Area Connection* 1:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Wireless LAN adapter Local Area Connection* 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

Wireless LAN adapter Wi-Fi:

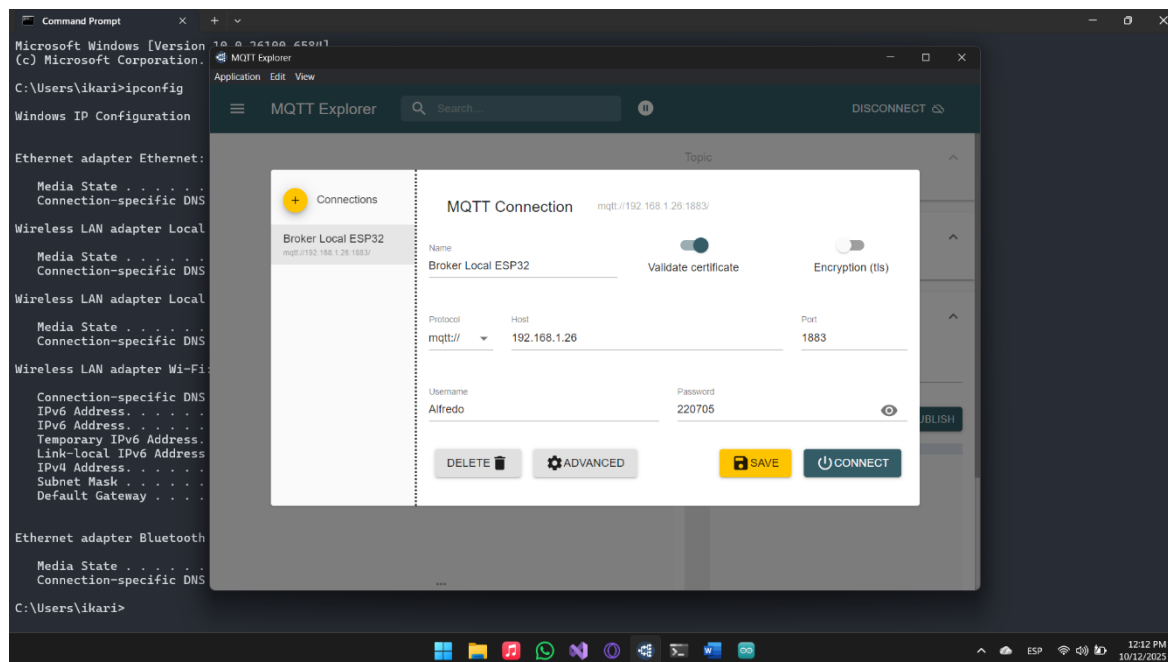
    Connection-specific DNS Suffix . :
    IPv6 Address. . . . . : 2803:2d60:110e:3e0::1
    IPv6 Address. . . . . : 2803:2d60:110e:3e0:8550:8b88:9e61:edae
    Temporary IPv6 Address. . . . : 2803:2d60:110e:3e0:25d5:13df:9c15:e114
    Link-local IPv6 Address . . . . : fe80:9e61:edae:9c15:993d%6
    IPv4 Address. . . . . : 192.168.1.26
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : fe80::763a:efff:fe19:9fff%6
                                192.168.1.1

Ethernet adapter Bluetooth Network Connection:

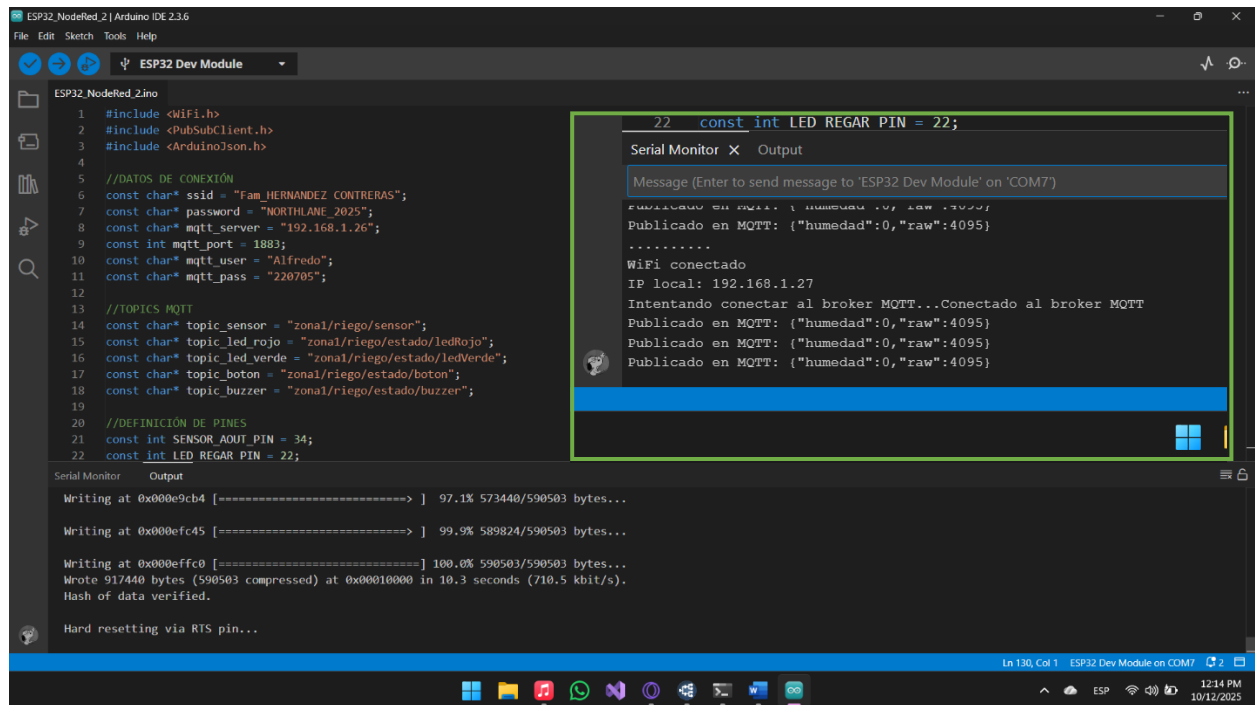
    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix . :

C:\Users\ikari>
```

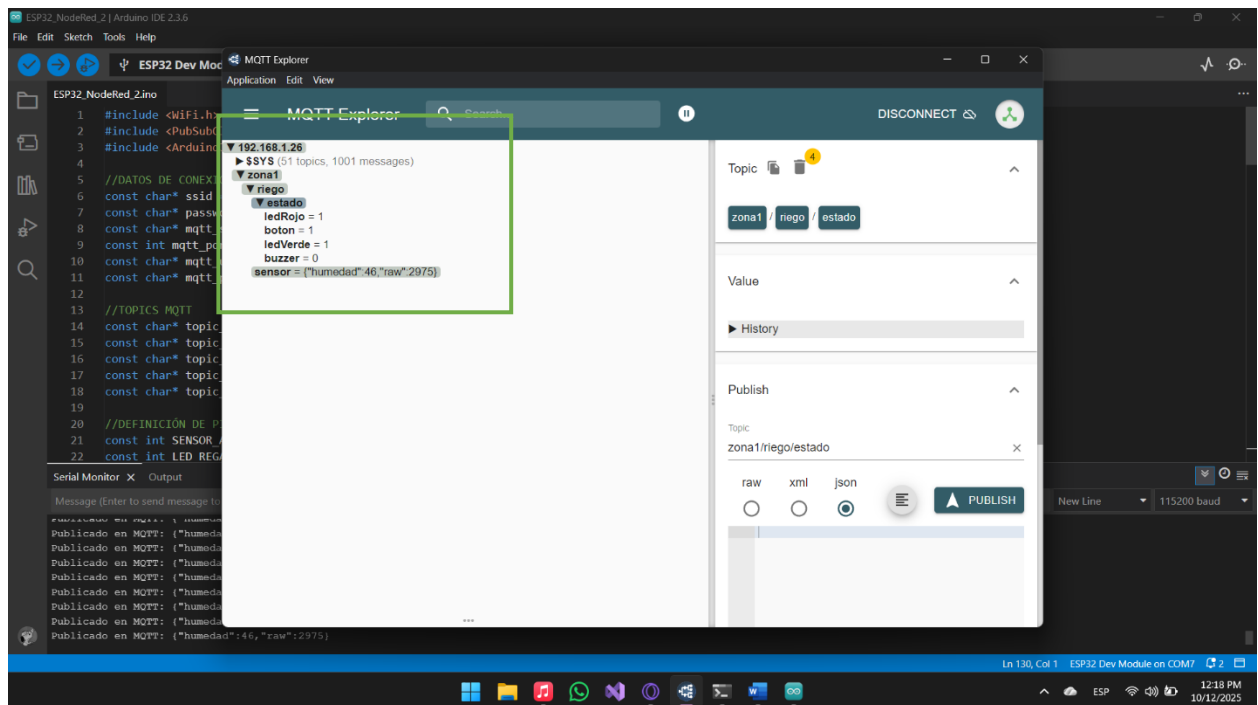
Se configura el MQTT Explorer con las credenciales y Host definidos.



Se compila el código al ESP32 y nos aseguramos la conexión a la red WiFi y al Bróker MQTT



Podemos visualizar en los Topics que están llegando variables reales del proceso, de esta manera confirmamos la conexión exitosa al bróker MQTT.



#### 4. Funcionamiento del proceso

El sistema de riego inteligente combina un montaje físico de sensores y actuadores con la lógica de control programada en el microcontrolador **ESP32**. Juntos, crean un sistema automatizado que reacciona a las condiciones del entorno y a la interacción del usuario.

El montaje físico interactúa directamente con nuestro entorno y nosotros, el usuario, en este caso el sensor de Humedad de suelo (**YL-69/FC-28**), actúa como los "dedos" del sistema. Al estar enterrado en la tierra, mide la resistencia eléctrica del suelo, la cual varía según la cantidad de agua presente. Envía esta información como una señal de voltaje analógica al ESP32.

Los LEDs (Rojo y Verde), son nuestros indicadores visuales. El LED rojo se enciende para alertar que la tierra está seca (**< 50%**), funcionando como una "luz de advertencia". El LED verde simula el estado de la bomba de riego, encendiéndose solo cuando el usuario presiona el botón, indicando que la "bomba está activa".

Y, por último, el pulsador, es la interfaz del usuario para la acción. Simula el encendido manual de una bomba de agua. Al mantenerlo presionado, el sistema entiende que se está realizando la acción de regar, en combinación de un buzzer, que funciona como una alarma sonora. Se activa cuando el sensor detecta un exceso de humedad (**> 80%**), alertando al usuario que debe detener el riego para evitar encharcamientos.

La ESP32 es el centro de control que procesa la información y toma todas las decisiones. Sus tareas principales, la de leer y traducir los datos que recibe constantemente, en este caso, la señal de voltaje del pin Aout del sensor de humedad. Usando su conversor analógico-digital (ADC), traduce este voltaje a un valor numérico (de **0 a 4095**). Luego, mediante una calibración (map), convierte este valor crudo en un porcentaje de humedad fácil de entender (**0% a 100%**).

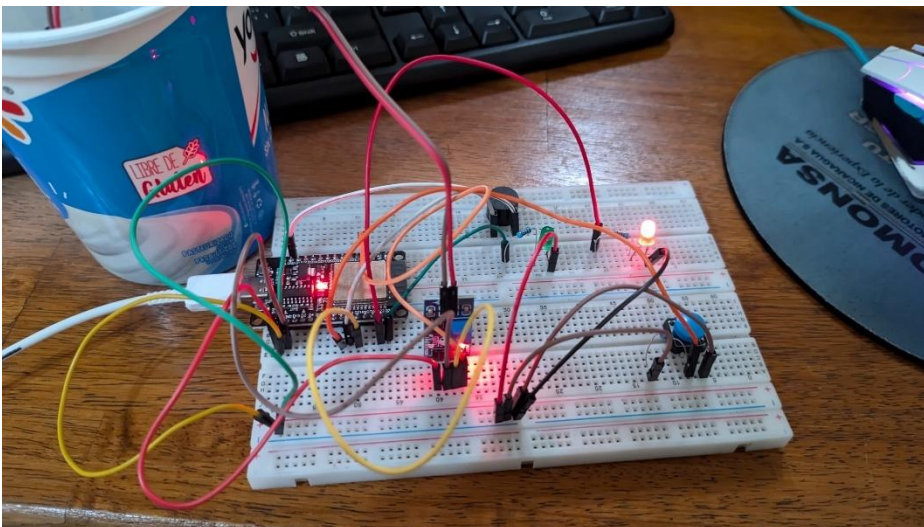
De esta manera, basándose en el porcentaje de humedad calculado, el ESP32 ejecuta una serie de condiciones if/else:

- ✓ Si la humedad es menor a 50%, envía una señal HIGH (encendido) al pin del LED rojo.
- ✓ Si la humedad supera el 80%, envía una señal HIGH al pin del buzzer.

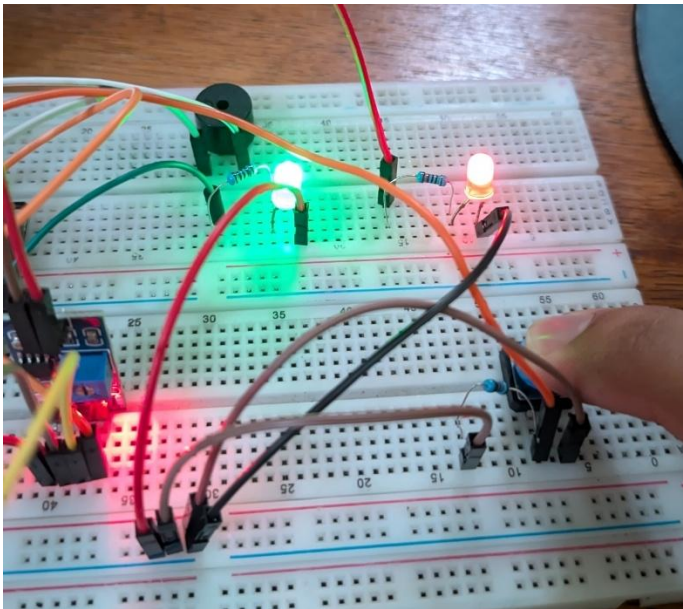
Independientemente de la humedad, si detecta una señal HIGH del pulsador, envía una señal HIGH al pin del LED verde.

Además, su función añade, el de comunicar el Estado (MQTT), actuando como un cliente de red. Se conecta a la red WiFi y al broker MQTT para:

- ✓ **Publicar Datos:** Envía periódicamente el porcentaje de humedad al topic **zona1/riego/sensor**.
- ✓ **Publicar Estados:** Cada vez que el estado de un LED, el buzzer o el pulsador cambia, publica un mensaje (**0 o 1**) en su topic correspondiente (ej. **zona1/riego/estado/ledRojo**). Esto permite que sistemas externos, como Node-RED, conozcan el estado exacto del sistema en tiempo real.



```
▼ estado
ledRojo = 1
boton = 0
ledVerde = 0
buzzer = 0
sensor = {"humedad":0,"raw":4095}
```



```

▼ Zona 1
▼ riego
  ▼ estado
    ledRojo = 1
    boton = 1
    ledVerde = 1
    buzzer = 0
    sensor = {"humedad":0,"raw":4095}

```

La comunicación de la ESP32 al bróker MQTT ha sido exitosa, podemos visualizar en los Topics lo que está pasando físicamente en nuestro montaje.

## 5. Configuración del Node-RED

Nos aseguramos de iniciar, instalar y conectar el node-red a nuestro bróker MQTT con los comandos ***npm install -g --unsafe-perm node-red*** en **Node.js Command Prompt**, luego de este paso, verificamos la instalación con ***node-red***

```

node-red
Your environment has been set up for using Node.js 22.20.0 (x64) and npm.

C:\Users\ikari>node-red
12 Oct 12:19:36 - [info]

Welcome to Node-RED
=====
12 Oct 12:19:36 - [info] Node-RED version: v4.1.0
12 Oct 12:19:36 - [info] Node.js version: v22.20.0
12 Oct 12:19:36 - [info] Windows_NT 10.0.26100 x64 LE
12 Oct 12:19:36 - [info] Loading palette nodes
12 Oct 12:19:38 - [info] Dashboard version 3.6.6 started at /ui
12 Oct 12:19:39 - [info] Settings file : C:\Users\ikari\.node-red\settings.js
12 Oct 12:19:39 - [info] Context store : 'default' [module=memory]
12 Oct 12:19:39 - [info] User directory : \Users\ikari\.node-red
12 Oct 12:19:39 - [warn] Projects disabled : editorTheme.projects.enabled=false
12 Oct 12:19:39 - [info] Flows file : \Users\ikari\.node-red\flows.json
12 Oct 12:19:39 - [info] Server now running at http://127.0.0.1:1880/
12 Oct 12:19:39 - [warn]

Your flow credentials file is encrypted using a system-generated key.

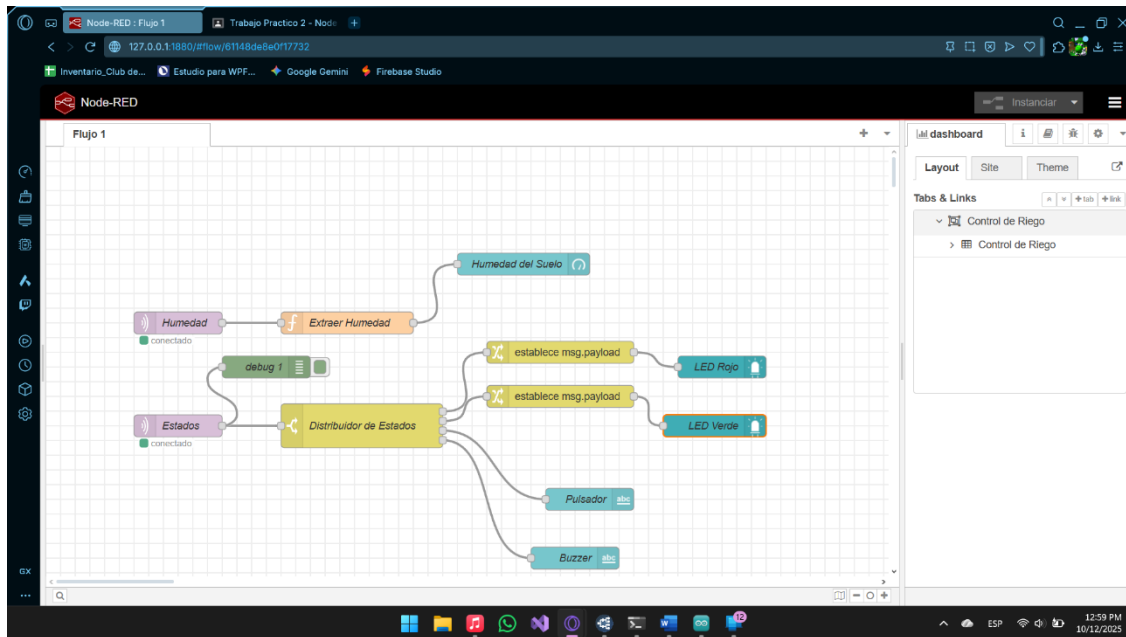
If the system-generated key is lost for any reason, your credentials
file will not be recoverable, you will have to delete it and re-enter
your credentials.

You should set your own key using the 'credentialSecret' option in
your settings file. Node-RED will then re-encrypt your credentials
file using your chosen key the next time you deploy a change.

12 Oct 12:19:39 - [info] Starting flows
12 Oct 12:19:39 - [info] Started flows
12 Oct 12:19:39 - [info] [mqtt-broker:Humedad] Connected to broker: mqtt://192.168.1.26:1883

```

Abrimos en el navegador la ventana principal de Node-RED <http://127.0.0.1:1880>



### Configuración de los Nodos:

#### 1. Nodo **mqtt in** (Humedad)

- ✓ Servidor: Añadimos nuestro bróker con la IP, puerto y credenciales.
- ✓ Topic: **zona1/riego/sensor**
- ✓ Salida: un objeto JSON

#### 2. Nodo **function** (Extraer Humedad)

- ✓ Código: **msg.payload = msg.payload.humedad; return msg;**
- ✓ Conéctalo a la salida del **mqtt in** de humedad.
- ✓ Nodo **gauge** (Medidor de Humedad)
- ✓ Rango: **Mín 0, Máx 100.**
- ✓ Etiqueta: Humedad del Suelo
- ✓ Conectamos a la salida del nodo **function**.

#### 3. Nodo **mqtt in** (Estados)

- ✓ Servidor: El mismo bróker.



✓ Topic: **zona1/riego/estado/#**

✓ Salida: una cadena

**4. Nodo `switch` (Distribuidor de Estados)**

✓ Propiedad: `msg.topic`

✓ Se configuran 4 reglas, cada una con la condición `==` y el tipo texto:

- **zona1/riego/estado/ledRojo**
- **zona1/riego/estado/ledVerde**
- **zona1/riego/estado/boton**
- **zona1/riego/estado/buzzer**

**5. Nodos de Visualización (Conectados a las salidas del switch)**

✓ Salida 1 -> Nodo **ui\_led** (LED Rojo)

- Etiqueta: Alerta: Regar
- Colores: 1 -> red, 0 -> grey

✓ Salida 2 -> Nodo **ui\_led** (LED Verde)

- Etiqueta: Riego Activo
- Colores: 1 -> green, 0 -> grey

✓ Salida 3 -> Nodo **ui\_text** (Pulsador)

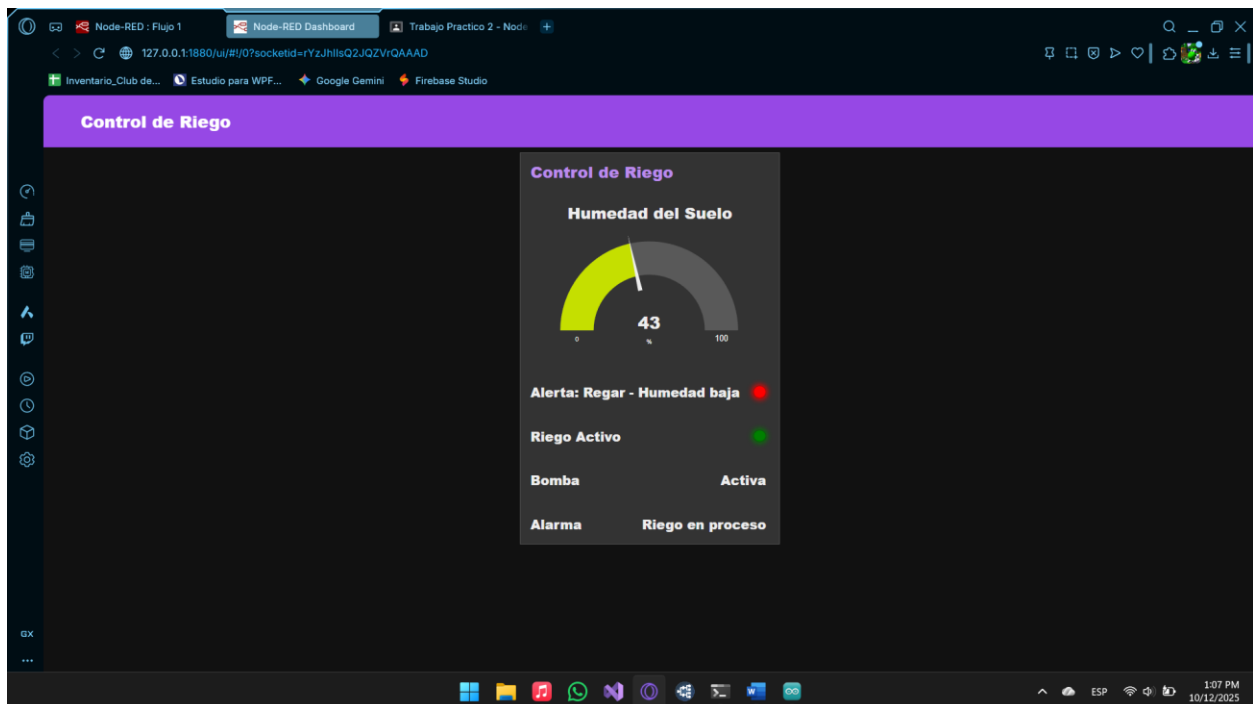
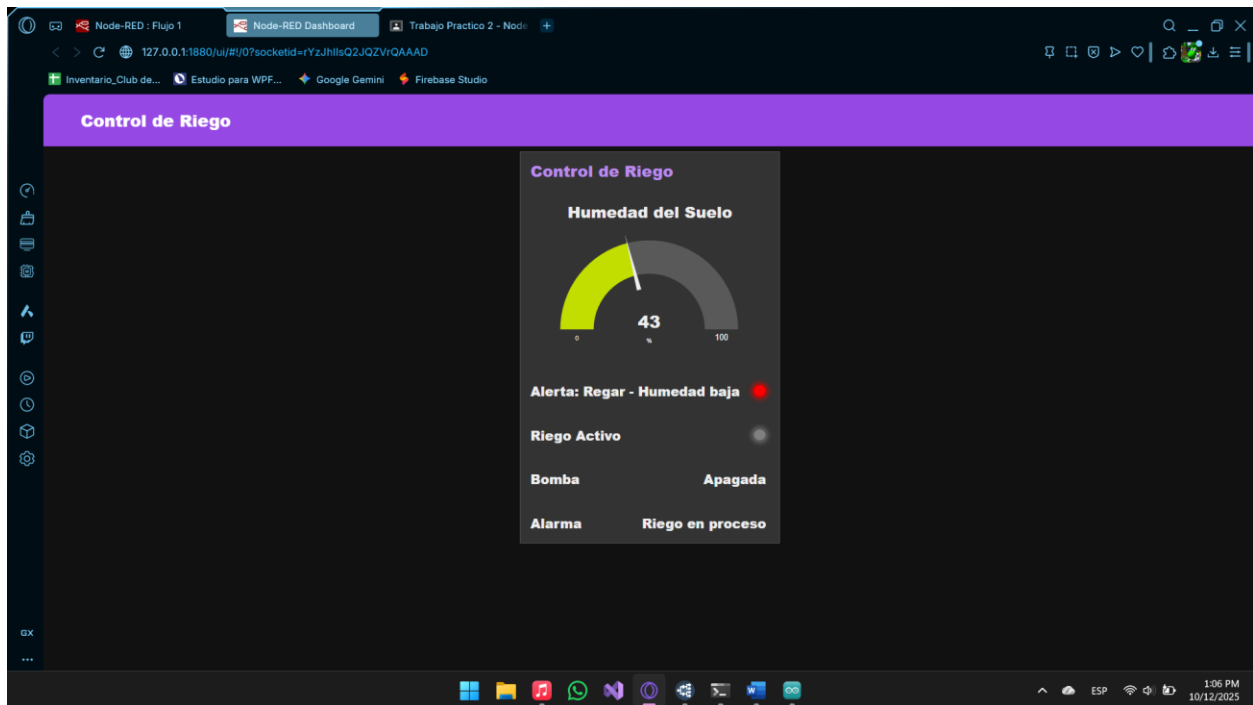
- Etiqueta: Pulsador\_Bomba
- Valor: `{{msg.payload == "1" ? "Activa" : "Apagada"}}`

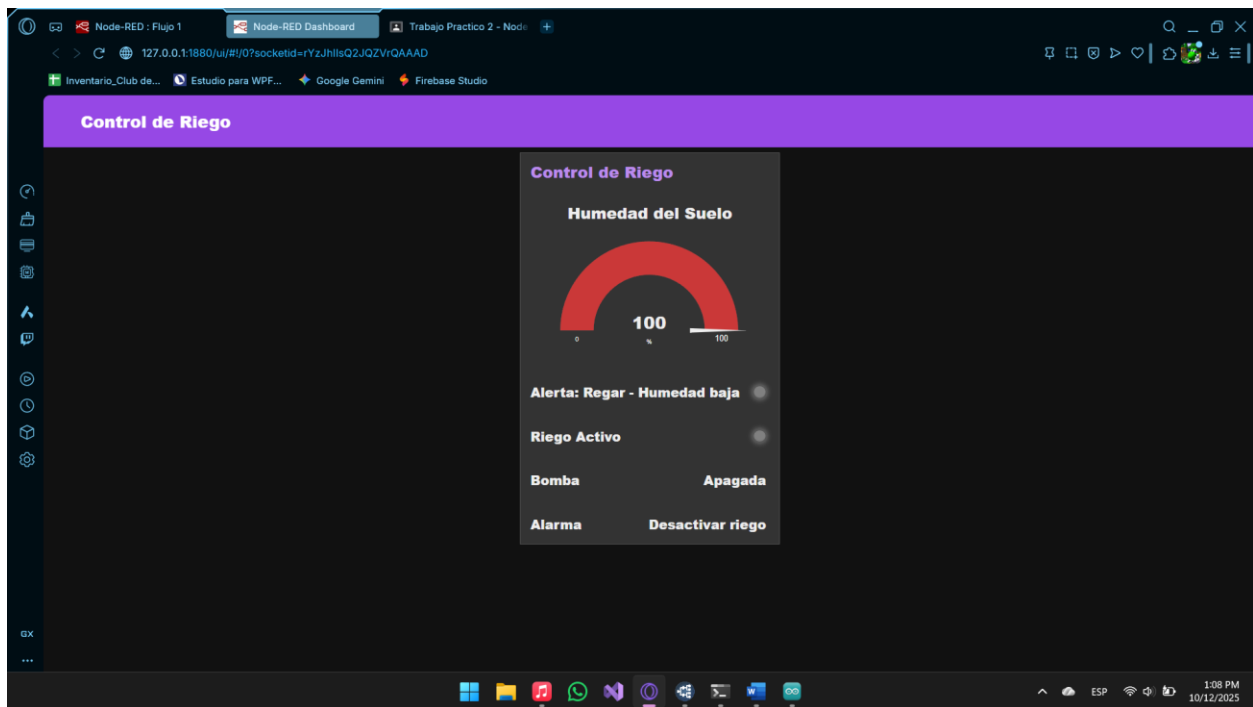
✓ Salida 4 -> Nodo **ui\_text** (Buzzer)

- Etiqueta: Buzzer
- Valor: `{{msg.payload == "1" ? "Desactivar riego" : "Riego en proceso"}}`



## Dashboard





#### 4. Resultados y Análisis

Una vez finalizada la integración de todos los componentes de hardware y software, el proyecto fue sometido a pruebas funcionales para validar su rendimiento y el cumplimiento de los objetivos establecidos. A continuación, se presentan los resultados obtenidos y un análisis de estos. El sistema demostró un funcionamiento robusto y coherente en sus tres áreas principales: control local, comunicación de datos y visualización remota.

##### 1. Operación del Sistema de Control Local

El circuito físico montado en la protoboard operó exactamente como fue diseñado. Las pruebas confirmaron que:

- El **sensor de humedad** proporcionó lecturas analógicas estables y consistentes, que el ESP32 tradujo correctamente a valores porcentuales.
- El **LED rojo de alerta** se activó de forma inmediata y fiable cuando el porcentaje de humedad descendió por debajo del umbral del 50%.

- El **pulsador** y el **LED verde** (simulador de bomba) funcionaron en perfecta sincronía; el LED se mantuvo encendido únicamente mientras el botón estuvo presionado.
- El **buzzer de alarma** se activó correctamente al superar el umbral del 80% de humedad, proporcionando una alerta sonora clara.

## 2. Rendimiento de la Comunicación MQTT

La transmisión de datos desde el ESP32 hacia el broker Mosquitto fue exitosa y eficiente.

- El ESP32 estableció una conexión estable con la red WiFi y el broker MQTT local, reconectándose automáticamente en caso de interrupción.
- Los datos del sensor se publicaron en formato JSON en el topic zona1/riego/sensor cada dos segundos, como se programó. Las verificaciones con MQTT Explorer mostraron que los payloads eran correctos y completos.
- Los cambios de estado de todos los actuadores (LEDs, buzzer y pulsador) se publicaron de forma **instantánea** en sus respectivos topics bajo zona1/riego/estado/. Esto confirma que la lógica de detección de cambios en el código del ESP32 funciona correctamente.

## 3. Visualización en el Dashboard de Node-RED

El dashboard final proveyó una interfaz de usuario clara, intuitiva y, lo más importante, sincronizada en tiempo real con el sistema físico.

- El **medidor (gauge)** de humedad reflejó con precisión y sin retraso perceptible las lecturas del sensor físico.
- Los **indicadores LED virtuales** en el dashboard cambiaron de color simultáneamente con los LEDs físicos.

- Los **nodos de texto** mostraron correctamente los estados "Presionado"/"Suelto" y "Sonando"/"Apagado", demostrando que el flujo de Node-RED (especialmente el nodo switch) estaba distribuyendo y formateando los mensajes de manera correcta.

## 6. Análisis

El éxito en los resultados obtenidos permite extraer varias conclusiones sobre la arquitectura y el diseño del proyecto.

### Robustez del Diseño de Doble Bucle

El sistema opera bajo un modelo de "doble bucle": un bucle de control local y un bucle de monitoreo remoto. Este es un diseño altamente robusto. El ESP32 es autónomo; si la conexión WiFi o el broker MQTT fallaran, las alertas y controles locales seguirían funcionando sin interrupción. Esto es crítico para aplicaciones que requieren alta fiabilidad.

### Eficiencia y Escalabilidad de MQTT

El protocolo MQTT demostró ser ideal para este tipo de aplicación IoT. Su bajo consumo de ancho de banda permitió actualizaciones casi instantáneas en el dashboard. La estrategia de usar topics jerárquicos y específicos (**zona1/riego/estado/ledRojo**, etc.) no solo facilitó la depuración, sino que también sienta las bases para una fácil escalabilidad. Añadir nuevos sensores o zonas de riego al sistema sería tan simple como definir nuevos topics, sin necesidad de modificar la lógica existente.

### Importancia Crítica de la Calibración

El análisis revela que la precisión de todo el sistema depende directamente de la calibración inicial del sensor de humedad. Los valores **VALOR\_SECO** y **VALOR\_HUMEDO** en el código son los pilares que dan sentido a las lecturas. Sin una calibración adecuada al tipo de suelo específico, los porcentajes mostrados serían inexactos, llevando a decisiones de riego incorrectas.

## 5. Conclusiones

Al finalizar este proyecto, se ha demostrado con éxito la viabilidad y eficacia de integrar tecnologías de hardware de bajo costo, protocolos de comunicación IoT y herramientas de programación visual para crear una solución robusta y funcional. El sistema de riego inteligente no solo cumplió con todos los objetivos planteados, sino que también sirvió como un profundo ejercicio práctico que evidencia el poder de la arquitectura de doble bucle, donde el control local autónomo se complementa perfectamente con el monitoreo remoto, garantizando una alta fiabilidad.

El protocolo MQTT se reafirmó como la columna vertebral de la comunicación, destacando por su ligereza y eficiencia. La capacidad del sistema para transmitir cambios de estado de manera instantánea desde el ESP32 al dashboard es un testimonio de la idoneidad de MQTT para aplicaciones de tiempo real. La estrategia de topics jerárquicos no solo simplificó el desarrollo y la depuración, sino que inherentemente dota al proyecto de una notable escalabilidad, permitiendo que futuras expansiones, como la adición de más sensores o zonas, se integren con un esfuerzo mínimo.

Node-RED, por su parte, demostró ser una herramienta excepcionalmente potente y accesible para la creación de interfaces de usuario. La capacidad de transformar flujos de datos complejos en un dashboard claro e intuitivo sin escribir extensas líneas de código es una de sus mayores fortalezas. La visualización lograda no es simplemente una réplica de datos, sino una interfaz de monitoreo funcional que permite a un usuario comprender el estado del sistema de un solo vistazo, cerrando así el ciclo entre el entorno físico y la interacción humana digital.

Reflexionando sobre el proceso, se destaca la importancia crítica de la calibración del sensor como un factor determinante en la precisión de todo el sistema. Este paso, aunque técnico, subraya que la inteligencia de cualquier sistema "inteligente" depende fundamentalmente de la calidad de sus datos de entrada. Asimismo, la experiencia de depuración, desde la verificación de la comunicación con MQTT Explorer hasta el análisis de flujos en Node-RED, reforzó la

importancia de contar con herramientas adecuadas para diagnosticar y resolver problemas en un sistema distribuido.

En conclusión, este proyecto trasciende la mera construcción de un circuito; representa una implementación exitosa de un ecosistema IoT completo, desde el sensor hasta la pantalla. Se ha creado una base sólida sobre la cual se pueden construir futuras innovaciones, como la integración de pronósticos del tiempo, algoritmos de aprendizaje automático para optimizar el riego, o la expansión a una red de sensores a gran escala. El conocimiento adquirido consolida nuestra comprensión de las tecnologías que están dando forma al futuro de la automatización y el control inteligente.

## 6. Anexos

```
[
  {
    "id": "61148de8e0f17732",
    "type": "tab",
    "label": "Flujo 1",
    "disabled": false,
    "info": "",
    "env": []
  },
  {
    "id": "1434e7023be32534",
    "type": "mqtt in",
    "z": "61148de8e0f17732",
    "name": "Humedad",
    "topic": "zona1/riego/sensor",
    "qos": "2",
    "datatype": "json",
    "broker": "c339b96035fb0106",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 180,
    "y": 240,
    "wires": [
      [
        "720557887a69cc5a"
```

```

    ]
  ],
  {
    "id": "720557887a69cc5a",
    "type": "function",
    "z": "61148de8e0f17732",
    "name": "Extraer Humedad",
    "func": "msg.payload = msg.payload.humedad;\nreturn msg;",
    "outputs": 1,
    "timeout": 0,
    "noerr": 0,
    "initialize": "",
    "finalize": "",
    "libs": [],
    "x": 410,
    "y": 240,
    "wires": [
      [
        "bd04b670a8f45dd8"
      ]
    ]
  },
  {
    "id": "bd04b670a8f45dd8",
    "type": "ui_gauge",
    "z": "61148de8e0f17732",
    "name": "Humedad del Suelo",
    "group": "3b4330b766f7d52c",
    "order": 0,
    "width": 0,
    "height": 0,
    "gtype": "gage",
    "title": "Humedad del Suelo",
    "label": "%",
    "format": "{{value}}",
    "min": 0,
    "max": "100",
    "colors": [
      "#00b500",
      "#e6e600",
      "#ca3838"
    ]
  },

```

```

    "seg1": "",
    "seg2": "",
    "diff": false,
    "className": "",
    "x": 650,
    "y": 160,
    "wires": []
  },
  {
    "id": "90d3f2e32edbe5ad",
    "type": "mqtt in",
    "z": "61148de8e0f17732",
    "name": "Estados",
    "topic": "zona1/riego/estado/#",
    "qos": "2",
    "datatype": "utf8",
    "broker": "c339b96035fb0106",
    "nl": false,
    "rap": true,
    "rh": 0,
    "inputs": 0,
    "x": 180,
    "y": 380,
    "wires": [
      [
        "2bbe13a1039608ed",
        "2d65ae2cc3c3d43e"
      ]
    ]
  },
  {
    "id": "2bbe13a1039608ed",
    "type": "switch",
    "z": "61148de8e0f17732",
    "name": "Distribuidor de Estados",
    "property": "topic",
    "propertyType": "msg",
    "rules": [
      {
        "t": "eq",
        "v": "zona1/riego/estado/ledRojo",
        "vt": "str"
      }
    ],

```



```
{
  {
    "t": "eq",
    "v": "zona1/riego/estado/ledVerde",
    "vt": "str"
  },
  {
    "t": "eq",
    "v": "zona1/riego/estado/boton",
    "vt": "str"
  },
  {
    "t": "eq",
    "v": "zona1/riego/estado/buzzer",
    "vt": "str"
  }
],
"checkall": "true",
"repair": false,
"outputs": 4,
"x": 430,
"y": 380,
"wires": [
  [
    "7d48a5db64b8f445"
  ],
  [
    "3ac7f1c75257d122"
  ],
  [
    "c5303d2af50bc08b"
  ],
  [
    "137284ad32a02b43"
  ]
]
},
{
  "id": "a954384744b2e444",
  "type": "ui_led",
  "z": "61148de8e0f17732",
  "order": 0,
  "group": "3b4330b766f7d52c",
  "width": 0,
```

```
"height": 0,
"label": "Alerta: Regar - Humedad baja",
"labelPlacement": "left",
"labelAlignment": "left",
"colorForValue": [
  {
    "color": "#ff0000",
    "value": "1",
    "valueType": "num"
  },
  {
    "color": "#808080",
    "value": "0",
    "valueType": "num"
  }
],
"allowColorForValueInMessage": false,
"shape": "circle",
"showGlow": true,
"name": "LED Rojo",
"x": 920,
"y": 300,
"wires": []
},
{
  "id": "43d8857a98ae3f08",
  "type": "ui_led",
  "z": "61148de8e0f17732",
  "order": 0,
  "group": "3b4330b766f7d52c",
  "width": 0,
  "height": 0,
  "label": "Riego Activo",
  "labelPlacement": "left",
  "labelAlignment": "left",
  "colorForValue": [
    {
      "color": "#008000",
      "value": "1",
      "valueType": "num"
    },
    {
      "color": "#808080",
```

```

        "value": "0",
        "valueType": "num"
    }
],
"allowColorForValueInMessage": false,
"shape": "circle",
"showGlow": true,
"name": "LED Verde",
"x": 910,
"y": 380,
"wires": []
},
{
    "id": "c5303d2af50bc08b",
    "type": "ui_text",
    "z": "61148de8e0f17732",
    "group": "3b4330b766f7d52c",
    "order": 0,
    "width": 0,
    "height": 0,
    "name": "Pulsador",
    "label": "Bomba ",
    "format": "{{msg.payload == \"1\" ? \"Activa\" : \"Apagada\"}}",
    "layout": "row-spread",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#000000",
    "x": 740,
    "y": 480,
    "wires": []
},
{
    "id": "137284ad32a02b43",
    "type": "ui_text",
    "z": "61148de8e0f17732",
    "group": "3b4330b766f7d52c",
    "order": 0,
    "width": 0,
    "height": 0,
    "name": "Buzzer",
    "label": "Alarma ",

```

```

    "format": "{{msg.payload == \"1\" ? \"Desactivar riego\" : \"Riego en proceso\"}}",
    "layout": "row-spread",
    "className": "",
    "style": false,
    "font": "",
    "fontSize": 16,
    "color": "#000000",
    "x": 720,
    "y": 560,
    "wires": []
  },
  {
    "id": "2d65ae2cc3c3d43e",
    "type": "debug",
    "z": "61148de8e0f17732",
    "name": "debug 1",
    "active": true,
    "tosidebar": true,
    "console": false,
    "tostatus": false,
    "complete": "payload",
    "targetType": "msg",
    "statusVal": "",
    "statusType": "auto",
    "x": 300,
    "y": 300,
    "wires": []
  },
  {
    "id": "7d48a5db64b8f445",
    "type": "change",
    "z": "61148de8e0f17732",
    "name": "",
    "rules": [
      {
        "t": "set",
        "p": "payload",
        "pt": "msg",
        "to": "$number(payload)",
        "tot": "jsonata"
      }
    ]
  },
],

```

```

    "action": "",
    "property": "",
    "from": "",
    "to": "",
    "reg": false,
    "x": 700,
    "y": 280,
    "wires": [
      [
        "a954384744b2e444"
      ]
    ]
  },
  {
    "id": "3ac7f1c75257d122",
    "type": "change",
    "z": "61148de8e0f17732",
    "name": "",
    "rules": [
      {
        "t": "set",
        "p": "payload",
        "pt": "msg",
        "to": "$number(payload)",
        "tot": "jsonata"
      }
    ],
    "action": "",
    "property": "",
    "from": "",
    "to": "",
    "reg": false,
    "x": 700,
    "y": 340,
    "wires": [
      [
        "43d8857a98ae3f08"
      ]
    ]
  },
  {
    {
      "id": "c339b96035fb0106",
      "type": "mqtt-broker",

```

```
"name": "Humedad",
"broker": "192.168.1.26",
"port": 1883,
"clientid": "",
"autoConnect": true,
"usetls": false,
"protocolVersion": 4,
"keepalive": 60,
"cleansession": true,
"autoUnsubscribe": true,
"birthTopic": "",
"birthQos": "0",
"birthRetain": "false",
"birthPayload": "",
"birthMsg": {},
"closeTopic": "",
"closeQos": "0",
"closeRetain": "false",
"closePayload": "",
"closeMsg": {},
"willTopic": "",
"willQos": "0",
"willRetain": "false",
"willPayload": "",
"willMsg": {},
"userProps": "",
"sessionExpiry": ""
},
{
  "id": "3b4330b766f7d52c",
  "type": "ui_group",
  "name": "Control de Riego",
  "tab": "9a302c65c239563e",
  "order": 1,
  "disp": true,
  "width": 6,
  "collapse": false,
  "className": ""
},
{
  "id": "9a302c65c239563e",
  "type": "ui_tab",
  "name": "Control de Riego",
```

```
    "icon": "dashboard",
    "disabled": false,
    "hidden": false
  },
  {
    "id": "81e7f8186b72412e",
    "type": "global-config",
    "env": [],
    "modules": {
      "node-red-dashboard": "3.6.6",
      "node-red-contrib-ui-led": "0.4.11"
    }
  }
]
```

## 7. Referencias bibliográficas

Grover, P., & Mehra, A. (2021). \*Internet of Things with ESP32\*. Packt Publishing.

Kadry, S., & Al-Taie, M. Z. (2022). \*IoT and Cloud Computing for Societal Good: Principles, Practices, and Applications\*. Springer International Publishing.

Light, A. (2016). \*MQTT Essentials: A Lightweight Messaging Protocol for the Internet of Things\*. O'Reilly Media, Inc.

O'Flaherty, D. (2021). \*Getting Started with Node-RED\*. Packt Publishing.

Panchal, P. (2022). \*Hands-On Internet of Things with MQTT: Build Connected IoT devices with MQTT and Node.js\*. BPB Publications.